Cheat Sheet and Sticky Notes

IaC principles, Terraform workflow, first AWS setup

> **Key idea.** Describe the **desired state** in configuration files. Terraform compares it with its known **state** and calls the **providers** to create, modify or delete the necessary resources.

Why IaC?

Benefit	Meaning	Benefit	Meaning
Automation	Fewer manual operations	Consistency	Same config across environments
Repeatability	Rebuild from a configuration	Traceability	Review changes before applying
Versioning	Track changes with Git	Collaboration	Share and review in a team

Imperative	Declarative (Terraform)
You list the **actions** and their order: create network, create VM, attach, add rule.	You state the **result**: a network, a VM on it, a rule allowing HTTP. The tool finds the steps.

Core components

Component	What it is
Terraform CLI	Command-line tool: initialize, check, plan, apply
Provider	Plugin to talk to a platform (AWS, Azure, GCP, Docker, Kubernetes, vSphere)
Resource	Object Terraform creates, modifies or deletes (`aws_instance`, `aws_security_group`, `aws_s3_bucket`)
Data Source	Reads info about an existing object, without necessarily creating it
State	File `terraform.tfstate`: records managed resources and maps config to real objects
HCL	HashiCorp Configuration Language, used in `.tf` files

Workflow

#	Command	Role	Remember
1	`terraform init`	Initialize, download providers, create `.terraform`	Run first
2	`terraform fmt`	Format files to Terraform conventions	Style only
3	`terraform validate`	Check syntax and structure	Not a cloud check
4	`terraform plan`	Preview changes (config vs state vs remote)	Changes nothing
5	`terraform apply`	Apply changes, asks confirmation	Read the plan
6	`terraform destroy`	Delete resources managed by the config and state	Real deletion

Anatomy of a resource block

```
resource "aws_instance" "web" {      # resource "<TYPE>" "<LOGICAL NAME>"
  ami           = "ami-xxxxxxxx"      # arguments
  instance_type = "t3.micro"
}
```

Type `aws_instance` (prefix `aws` = provider), logical name `web`.

First setup (Windows + AWS)

```
./terraform.exe --version                          # check install
aws configure --profile terraform-lab               # key ID, secret, region, output
aws configure list --profile terraform-lab          # verify the profile
aws sts get-caller-identity --profile terraform-lab # verify the identity
```

```
# provider.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "6.72.0"
    }
  }
}

provider "aws" {
  region  = "us-east-1"
  profile = "terraform-lab"
}
```

```
# Path fix for the current PowerShell session
$env:Path += ";C:\Users\Admin\Downloads\terraform_1.17.1_windows_amd64"
```

> **Warning.** `terraform destroy` deletes real cloud resources. Check the plan and the targeted environment before confirming. Never publish `terraform.tfstate` in a public repository.

Exam traps

- **validate** passing does not mean AWS will accept the resources.
- **plan** never modifies anything; **apply** does.
- **Resource** is managed by Terraform; **data source** only reads.
- Terraform is mainly **declarative**, not imperative.
- The **provider** is deduced from the resource type prefix (aws_...).
- **init** comes first: without it, no providers are downloaded.

Sticky notes

init first

Run terraform init first. It downloads the providers and creates the .terraform folder.

plan = preview

plan changes nothing. It compares config, state and real infrastructure, then shows the diff.

apply = do it

apply executes the changes after you confirm. Always read the plan first.

validate is shallow

validate checks syntax and structure only. It does NOT guarantee AWS can create the resource.

destroy = real deletion

Deletes real cloud resources. Check the plan and the target environment before confirming.

state is sensitive

terraform.tfstate maps config to real objects and may hold secrets. Never push it to a public repo.

declarative

Describe the WANTED result. Terraform works out the steps. Imperative = list the actions.

provider = plugin

Lets Terraform talk to a platform: AWS, Azure, GCP, Docker, Kubernetes, vSphere.

resource vs data

Resource: Terraform manages it. Data source: only reads something that already exists.

read the block

resource "aws_instance" "web": type = aws_instance, name = web, provider = aws.

AWS profile

aws configure --profile terraform-lab, then profile = "terraform-lab" in provider.tf.

PATH error?

Add the Terraform folder to Path. For the current PowerShell session: $env:Path += ";folder"